A Two-Stage Entity Resolution Pipeline with FAISS Blocking and Splink Matching

Denis Robert
Independent Researcher

August 2026

Abstract

This paper presents and evaluates a two-stage entity resolution pipeline for incomplete Canadian person records. A synthetic reference population is represented with normalized `all-MiniLM-L6-v2` embeddings and persisted in a FAISS `IndexFlatIP` store. Each query is first blocked to its top 20 vector neighbors, after which Splink performs interpretable probabilistic linkage using Jaro–Winkler comparisons for names, email, and address, together with exact date-of-birth comparison. The approach is similar in overall architecture to the FAISS-plus-linkage workflow described by Strojny and Bereşewicz [6], although the present pipeline was independently derived and uses a different implementation and evaluation design. The implementation supports persisted-index reload without re-embedding the reference population and independently models 30% missingness for address and email. In a 15,000-query experiment comprising identical, close-variation, and unrelated records, the pipeline achieved 99.41% accuracy, 99.57% precision, 99.55% recall, 99.14% specificity, and a 99.56% F1 score, with a mean query time of 7.26 ms in the validation environment. The paper also discusses memory scaling, operational limits, calibration of Splink parameters, and alternatives to linear-scan FAISS indexing.

1 Introduction

Entity resolution attempts to determine whether two records refer to the same real-world entity. This task is difficult when records contain typographical variation, missing values, stale addresses, or inconsistent contact information. Comparing every query against every reference record is also unnecessarily expensive: for N reference records, exhaustive comparison requires $O(N)$ candidate comparisons per query.

The implementation in the entity-resolution repository addresses this problem with a retrieval-and-linkage architecture. A dense vector index performs approximate or exact nearest-neighbor retrieval, reducing the candidate set to the closest k records. This use of dense retrieval for blocking follows a line of work including DeepER [5], pre-trained embedding comparisons [4], and universal dense blocking [3]. Splink then evaluates those candidates with explicit field comparisons and returns a match only when its probability exceeds a configurable threshold.

The representation choice is informed by recent work comparing tabular representation approaches. Vogel et al.'s *Towards Universal Tabular Embeddings: A Benchmark Across Data Tasks* (arXiv:2604.21696v1, `https://arxiv.org/abs/2604.21696v1`) [8] benchmark embeddings at cell, row, column, and table levels and show that performance depends on the downstream task and representation level; in the relevant current evaluation, textual embeddings outperform the dedicated tabular alternatives for retrieval. Franz et al.'s *Universal Embeddings of Tabular Data*

(arXiv:2507.05904v1, https://arxiv.org/abs/2507.05904v1) [7] describes a different strategy that transforms tabular data into a graph, learns entity embeddings with graph auto-encoders, and aggregates them into row embeddings. This graph-based approach may bear fruit for entity resolution in future experiments because it is designed to capture relationships among tabular entities and to embed unseen samples composed of similar entities. The present system therefore uses a row-level textual representation as its current baseline while leaving room to replace the embedding engine as these alternatives become more capable.

2 System Objective and Data Model

The reference population contains 50,000 synthetic Canadian person records. Each record has:

- first name;
- last name;
- date of birth;
- a Canadian address; and
- an email address.

Address and email are independently removed with a configurable default probability of 0.30. Consequently, first name, last name, and date of birth are the most consistently available identity attributes, while address and email are useful but incomplete evidence. Addresses are also treated as less stable because people move and formatting conventions change.

A record is serialized into a structured text string:

```
First Name: ...
Last Name: ...
Date of Birth: ...
Address: ...
Email: ...
```

Missing fields are omitted from the embedding text and retained as null metadata. The same person object is used for the query and for the persisted metadata, avoiding a second source of truth.

2.1 Row serialization strategies

The embedding ablation compares three textual representations of the same structured record. These representations affect only dense retrieval; Splink receives the original structured fields and uses the same comparisons in every experiment. For example, consider the record

```
first_name    = Alice
last_name     = Wong
date_of_birth = 1985-06-15
address       = 123 Main Street, Toronto, ON M5V 1A1
email         = alice.wong@example.com
```

The **default** strategy is the production representation implemented by `Person.to_text`. It uses labelled lines in identity, address, and email order:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Address: 123 Main Street, Toronto, ON M5V 1A1
Email: alice.wong@example.com
```

The **identity-first** strategy retains the same labels and values but places the optional contact fields in email-then-address order after the identity fields:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Email: alice.wong@example.com
Address: 123 Main Street, Toronto, ON M5V 1A1
```

This tests whether giving the embedding model an explicit identity prefix changes retrieval, without changing the vocabulary or the underlying data. The **compact** strategy removes labels and joins the fixed field sequence with pipe delimiters:

```
Alice|Wong|1985-06-15|123 Main Street, Toronto, ON M5V 1A1|alice.wong@example.com
```

For a record with missing address and email, the labelled strategies omit those lines, whereas compact serialization preserves field positions with empty values:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
```

```
Alice|Wong|1985-06-15||
```

The default strategy is the baseline used by the resolver and confusion-matrix experiment. Identity-first and compact are evaluation alternatives, not separate linkage models. Comparing them under the same FAISS index type, blocking sizes, Splink configuration, queries, and thresholds isolates the effect of row representation on candidate recall and downstream matching.

3 Two-Stage Algorithm

3.1 Offline generation and indexing

The offline process in `generate_data.py` performs the following steps:

1. Generate the reference population with a fixed random seed when reproducibility is required.
2. Convert every person into its textual row representation.
3. Compute a dense vector with `sentence-transformers/all-MiniLM-L6-v2` through `HuggingFaceEmbeddings`.
4. L2-normalize each vector and add it to a FAISS `IndexFlatIP` index. Inner product on normalized vectors is cosine similarity.

5. Persist the FAISS index as `people.faiss` and the person records, model name, and normalization setting as `people.json`.

The metadata file is deliberately separate from the FAISS binary index. FAISS stores the vectors and their positional order; the JSON file stores the record associated with each position. Loading reconstructs the embedding client and reuses the saved vectors without re-embedding the reference population.

3.2 Online blocking

Given a query record q, the online process embeds the same textual representation and searches the index for the top k records. The default is $k = 20$. If $e(q)$ and $e(r)$ are normalized query and reference vectors, the blocking score is

$$s_{\text{FAISS}}(q, r) = e(q)^T e(r),$$

which is cosine similarity in the range $[-1, 1]$. This stage is optimized for recall: the true match must be included in the candidate set for the second stage to recover it.

3.3 Probabilistic linkage

The candidate set is passed to Splink as a small deduplication problem containing the query and the retrieved records. Splink implements a scalable form of probabilistic linkage grounded in the Fellegi–Sunter framework [1, 2]. The configured comparisons are:

Table 1: Field comparisons and intended evidence priority.

Field	Comparison	Intended role
First name	Jaro–Winkler thresholds	High-priority identity evidence
Last name	Jaro–Winkler thresholds	High-priority identity evidence
Date of birth	Exact match	Strong, stable evidence
Email	Jaro–Winkler thresholds	Medium-priority evidence when present
Address	Jaro–Winkler thresholds	Lower-priority, changeable evidence

Splink produces a match probability from the comparison vector and its m/u parameters. The resolver selects the highest-scoring candidate involving the query and returns it only when

$$P(\text{match} \mid \text{comparisons}) \geq \tau,$$

where τ is the configured match threshold. Missing values are not treated as negative evidence; they provide no comparison agreement and the remaining fields determine the result.

The implementation uses FAISS for blocking and Splink 4.x inference for linkage. The current settings provide comparison ordering and thresholded similarity levels. In a production deployment, Splink's m/u probabilities should be trained on representative labelled pairs rather than relying on defaults.

4 Persistence and Operational Workflow

The workflow has two explicit commands:

```
python generate_data.py --count 50000 --missing-rate 0.3 --output-dir data
python main.py --index-dir data --input-json query.json --threshold 0.85
```

The first command is an offline or scheduled job. The second command loads the persisted index, retrieves 20 candidates, performs Splink linkage, and emits either a match with both scores and the matched metadata or a no-match decision. This separation prevents expensive population embedding from occurring for every query and makes the index a deployable artifact.

5 FAISS Applicability and Memory Bounds

The 50,000-record artifact generated by this implementation provides a concrete sizing baseline. On disk, the measured `people.faiss` file is 76.8 MB and the `people.json` metadata file is 10.5 MB, for a combined 87.3 MB. This corresponds to approximately 1,536 bytes per normalized 384-dimensional float32 vector, 210 bytes per serialized person record, and 1,746 bytes per record including both artifacts. The index dimension follows the MiniLM embedding model used by the baseline.

The following extrapolation assumes the same embedding dimension, float32 storage, flat inner-product index, average metadata length, and one metadata record per vector. It uses binary RAM units (1 GiB = 2^{30} bytes), although the table labels them as the commonly used 16, 32, and 128 GB deployment sizes. It excludes the Python interpreter, embedding model, temporary construction buffers, allocator fragmentation, operating-system memory, and concurrent queries.

Table 2: Estimated capacity when the FAISS index and person metadata share the available RAM.

Available RAM	Raw capacity	Raw index size	Raw metadata size	Recommended capacity*
16 GB	9.84 million	14.4 GB	2.0 GB	6.89 million
32 GB	19.68 million	28.9 GB	4.1 GB	13.77 million
128 GB	78.71 million	115.3 GB	16.5 GB	55.10 million

*Recommended capacity reserves 30% of RAM for the operating system, Python process, model weights, FAISS temporary allocations, and query concurrency. The raw capacity is a sizing upper bound, not a production recommendation. During generation, peak memory can be higher because embeddings may be materialized before being added to FAISS. Building large indexes should therefore use batches and measure peak resident set size rather than relying only on final file size.

For this flat `IndexFlatIP` design, a query scans all vectors, so memory capacity and query latency are coupled to the record count. At tens of millions of records, an approximate index such as HNSW or an inverted-file/product-quantized index should be evaluated. Quantization can reduce memory substantially, but may reduce blocking recall and must be measured against the downstream Splink match rate. Regardless of the FAISS index type, the metadata mapping must preserve vector-to-person row order and should be loaded with equivalent memory budgeting.

5.1 When an external vector database becomes appropriate

An external vector database is not automatically better than FAISS. For a single service instance, a stable reference population, and a dataset that fits comfortably in memory, local FAISS has fewer moving parts and avoids network latency. The decision should be based on operational bounds as well as raw capacity. With the measured baseline, 10 million records would require approximately 17.5 GB for the flat index plus metadata, while 50 million records would require approximately

87.3 GB before runtime overhead. These sizes place a single-process deployment near or beyond the recommended envelope of a 16 or 32 GB host and make a 128 GB host increasingly specialized.

An external vector database is likely to become more appropriate under any of the following conditions:

- the working set approaches roughly 70% of host RAM, leaving insufficient room for the embedding model, Python, Splink, index rebuilds, and concurrent queries;
- the reference population is large enough that flat-index scan latency or index-build time violates the service-level objective, even after testing FAISS approximate indexes;
- multiple application instances need a shared index, replicas, rolling updates, or failover without copying a large binary artifact to every host;
- records require frequent inserts, deletes, or metadata updates, making immutable local snapshots operationally expensive; or
- the system needs managed durability, access control, monitoring, backups, multi-tenant isolation, or independent scaling of storage and query capacity.

As a practical boundary, local FAISS is a strong default for the 50,000-record baseline and remains straightforward through the low millions. Between roughly 7 million and 14 million records, the 16/32 GB sizing envelope suggests benchmarking an approximate FAISS index against a managed or self-hosted vector database. Above roughly 50 million records, or whenever the index must be shared across services and regions, an external system is generally the safer operational choice. These are planning thresholds rather than algorithmic limits: compression, sharding, and hardware can move them, while a strict latency or availability requirement can move the decision earlier. The external database should still be evaluated by blocking recall at $k = 20$, not only by vector-query latency, because Splink cannot recover a true match that the retrieval layer omits.

6 Complexity and Failure Modes

Let d be the embedding dimension, N the reference population size, and k the blocking size. Building the flat FAISS index costs approximately $O(Nd)$ vector storage and embedding work. A flat inner-product query costs $O(Nd)$, although FAISS offers approximate indexes when the population grows beyond the intended scale. Splink then operates on only $k + 1$ records, making the linkage stage independent of the full population size after blocking.

The principal failure mode is blocking recall. A correct record omitted from the top k candidates cannot be recovered by Splink. Other risks include embedding representations that overemphasize address tokens, duplicate or common names, poorly calibrated Splink probabilities, and distribution shift between synthetic data and operational data. Evaluation should therefore report blocking recall separately from final precision, recall, and calibration.

7 Evaluation Plan

A useful benchmark should contain labelled positive and negative pairs, controlled missingness, realistic address changes, spelling errors, and hard negatives sharing names or dates. The following metrics should be reported:

1. blocking recall at $k \in \{10, 20, 50, 100\}$;

2. final precision, recall, F1, and false match rate at several thresholds;

3. precision–recall and calibration curves for Splink probabilities;

4. mean and tail query latency; and

5. index size and offline build time.

The benchmark should include ablations for missing email, missing address, both fields missing, address changes, and name perturbations. It should also compare row serialization strategies so that improvements are attributable to representations rather than only to the matcher.

7.1 Measured Section 7 benchmark

These requirements were implemented in `evaluate_section7.py` and run with 5,000 reference records, 15,000 labelled queries, a 30% independent missingness rate, and 500 records per ablation. The labelled queries consisted of identical positives, close same-entity variants, and unrelated negatives. Splink used its current untrained default m/u parameters, so the measurements are an engineering baseline rather than calibrated production probabilities. The evaluator wrote the complete results to `section7_results.json` and `section7_metrics.csv`.

Table 3 compares the three row serialization strategies. Blocking recall is measured over the 10,000 positive queries. Latency percentiles cover embedding plus FAISS blocking per query; the reported end-to-end mean adds the batched Splink inference time divided by the number of queries.

Table 3: Measured serialization and retrieval/linkage results.

Strategy	R@10	R@20	R@50	R@100	$P_{.85}$	$R_{.85}$	$F1_{.85}$
Default	99.86%	99.88%	99.92%	99.94%	99.71%	99.93%	99.82%
Identity first	99.87%	99.88%	99.92%	99.93%	99.71%	99.92%	99.82%
Compact	99.94%	99.94%	99.97%	99.98%	99.73%	99.95%	99.84%

The compact serialization was the strongest of the three in this run, improving top-20 blocking recall from 99.88% to 99.94% and reducing the false-match rate at threshold 0.85 from 0.58% to 0.54%. The difference is descriptive rather than statistically conclusive and should be retested on labelled operational data. For the default strategy, index build time was 34.39 seconds, p95 blocking latency was 23.26 ms, and estimated mean end-to-end latency was 21.08 ms. The corresponding values for identity-first were 39.30 seconds, 26.88 ms, and 24.39 ms; compact was 34.93 seconds, 22.50 ms, and 21.46 ms.

Table 4: Blocking-recall ablations at $n = 500$ per condition.

Condition	R@10	R@20	R@50	R@100
Baseline	100.0%	100.0%	100.0%	100.0%
Missing email	100.0%	100.0%	100.0%	100.0%
Missing address	99.0%	99.4%	99.8%	100.0%
Missing both	98.0%	99.2%	99.8%	100.0%
Address change	100.0%	100.0%	100.0%	100.0%
Name perturbation	100.0%	100.0%	100.0%	100.0%

At the default 0.85 threshold, every ablation query was accepted in this synthetic positive-only slice once it reached Splink, so the main observed degradation was retrieval: removing both optional

fields reduced top-10 recall to 98.0% and top-20 recall to 99.2%. The evaluator also produces ten-bin reliability data and threshold curves; those outputs should be recalculated after Splink is trained on labelled pairs.

8 Confusion-Matrix Experiment

The implementation includes `test_confusion_matrix.py`, which evaluates the current algorithm on $n = 5{,}000$ generated reference rows. For every reference person, the test creates three queries: an identical row labelled as the same entity, a close but non-identical row generated with controlled perturbations and labelled as the same entity, and an independently generated unrelated row labelled as a different entity. The latter is not present in the reference index. FAISS retrieves the top 20 candidates for every query, after which Splink applies the same field comparisons and decision threshold used by the resolver. The experiment batches the queries into one Splink link-only inference call for execution efficiency; this changes execution strategy but not the candidate pairs or scoring configuration.

Table 5: Confusion-matrix experiment metaparameters.

Parameter	Value
Reference/test rows (n)	5,000
Queries per test row	3
Total queries	15,000
Missing address/email rate	0.30
Embedding model	`all-MiniLM-L6-v2`
FAISS blocking size (k)	20
Splink match threshold (τ)	0.85
Close-row variation rate	0.15
Random seed	42

Table 6: Measured confusion matrix over 15,000 queries.

	Predicted match	Predicted no match
Actual same entity	9,955 (TP)	45 (FN)
Actual different entity	43 (FP)	4,957 (TN)

All 5,000 identical queries were correctly matched. Among the 5,000 close same-entity queries, 4,955 were matched and 45 were missed. Among the 5,000 unrelated queries, 43 were incorrectly accepted and 4,957 were rejected. The resulting accuracy was 99.41%, precision 99.57%, recall 99.55%, specificity 99.14%, and F1 99.56%. Index construction took 31.5 seconds and batched query processing took 108.9 seconds, or 7.3 milliseconds per query on the test environment. These measurements are a baseline rather than a general performance guarantee; hardware, model caching, and Splink configuration affect both quality and latency.

9 Further Research

9.1 Back-propagation through the linkage pipeline

Splink is primarily a probabilistic SQL-based linkage engine, while FAISS retrieval and discrete comparison levels are not directly differentiable. Nevertheless, its metaparameters can be optimized with a differentiable surrogate. Let θ represent learnable parameters such as comparison thresholds, field weights, missing-value handling, and the final decision threshold. For a labelled pair $y \in \{0, 1\}$, define a soft comparison vector $c_\theta(q, r)$ and a differentiable match score $p_\theta(q, r)$. A supervised objective is

$$\mathcal{L}(\theta) = -y \log p_\theta(q, r) - (1 - y) \log(1 - p_\theta(q, r)) + \lambda \Omega(\theta).$$

A practical research path is:

1. export Splink comparison features and agreement levels for labelled pairs;
2. replace hard thresholds with temperature-controlled soft gates, such as a sigmoid over Jaro–Winkler distance;
3. learn field weights and soft m/u parameters with gradient descent;
4. distill the learned scoring function back into Splink-compatible comparison levels or calibrated weights; and
5. evaluate whether the distilled model preserves Splink's interpretability and SQL execution properties.

End-to-end back-propagation through top-k FAISS selection is harder because nearest-neighbor selection is discrete. Candidate retrieval can initially be held fixed while optimizing Splink parameters. Later experiments can use a soft top-k relaxation, hard-negative mining, or a separate contrastive loss for the embedding model. The system should retain a non-differentiable production path if the differentiable surrogate is only used for training.

9.2 Alternative embedding engines

The current MiniLM encoder is a useful baseline, but it is not specialized for entity resolution or structured rows. Candidate alternatives include:

- larger sentence-transformer models for stronger semantic representations;
- multilingual encoders for names and addresses across language communities;
- character n-gram or TF–IDF vectors for typo-sensitive retrieval;
- domain-adapted contrastive encoders trained on positive and hard-negative person pairs;
- graph-based universal tabular embeddings such as those proposed by Franz et al. [7]; and
- hybrid retrieval that combines dense similarity with lexical or field-specific indexes.

The comparison should measure not only final linkage quality but also whether the embedding preserves the true match in the top 20. A particularly promising approach is multi-view retrieval: learn separate views for identity fields, contact fields, and address fields, then combine their scores before Splink. This could reduce the tendency of a long address to dominate a short but highly identifying date of birth.

9.3 Calibration and deployment

Future work should estimate Splink parameters from labelled operational samples, quantify probability calibration, and introduce drift monitoring for names, domains, postal formats, and address conventions. Privacy-preserving evaluation is also important: synthetic data is useful for development, but production validation must control access to personally identifying information and log only the minimum necessary evidence.

10 Conclusion

The implemented architecture combines a fast vector retrieval layer with an interpretable probabilistic linkage layer. FAISS narrows a 50,000-record population to 20 candidates, while Splink compares identity, contact, and address fields under a configurable decision threshold. Persisting the index and metadata makes the workflow operationally reusable. The most important next steps are to train and calibrate Splink parameters on labelled pairs, investigate differentiable surrogates for its metaparameters, and benchmark alternative row and tabular embedding engines using blocking recall as a first-class metric.

References

[1] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. Journal of the American Statistical Association, 64(328):1183–1210, 1969. https://doi.org/10.1080/01621459.1969.10501049.

[2] Robin Linacre, Sam Lindsay, Theodore Manassis, Zoe Slade, Tom Hepworth, Ross Kennedy, and Andrew Bond. Splink: Free software for probabilistic record linkage at scale. International Journal of Population Data Science, 7(3), 2022. https://doi.org/10.23889/ijpds.v7i3.1794. Software repository: https://github.com/moj-analytical-services/splink.

[3] Tianyi Wang, Hongyu Lin, Xu Han, Xu Chen, Bo Cao, and Liang Sun. Towards universal dense blocking for entity resolution. 2024. Code repository: https://github.com/tshu-w/uniblocker.

[4] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. Pre-trained embeddings for entity resolution: An experimental analysis. Proceedings of the VLDB Endowment, 16(9):2225–2238, 2023. https://doi.org/10.14778/3598581.3598594.

[5] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. Proceedings of the VLDB Endowment, 11(11):1454–1467, 2018. https://doi.org/10.14778/3236187.3236198.

[6] Tymoteusz Strojny and Maciej Beręsewicz. BlockingPy: Approximate nearest neighbours for blocking of records for entity resolution. SoftwareX, 34:102583, 2026. https://doi.org/10.1016/j.softx.2026.102583. Software repository: https://github.com/ncn-foreigners/BlockingPy.

[7] Astrid Franz, Frederik Hoppe, Marianne Michaelis, and Udo Göbel. Universal embeddings of tabular data. arXiv:2507.05904v1, 2025. https://arxiv.org/abs/2507.05904v1.

[8] Liane Vogel, Kavitha Srinivas, Niharika D'Souza, Sola Shirai, Oktie Hassanzadeh, and Horst Samulowitz. Towards universal tabular embeddings: A benchmark across data tasks. arXiv:2604.21696v1, 2026. https://arxiv.org/abs/2604.21696v1.